

CODING GUIDELINES FOR DEVELOPMENT

JASMIN INFOTECH PVT. LTD.
No 119 , Velachery Tambaram Road,
Pallikaranai,
Chennai 601302

Document history:

S.No	Version No.	Author	Change(s) made	Approved Date	Approved by
1	1.0	Jasmin	First Release	10 / 10 / 2003	Jasmin
2	1.1	Jasmin	Feedback from various teams incorporated	27 / 10 / 2003	Jasmin
3	1.2	Jasmin	General requirements of decoders added considering new automotive ROM development	24 / 06 / 2005	Jasmin
4	1.3	Jasmin	General requirements of decoders elaborated	30 / 08 / 2005	Jasmin
5	2.0	Lakshmanan.M Prabhu.J	Guidelines document is fully modified.	12 / 06 / 2007	Senthil.A
6	2.1	Lakshmanan.M Wahidennisa.R	Assembly coding guidelines modified	11 / 12 / 2007	Senthil.A

TABLE OF CONTENTS

1	Purpose	5
2	Naming Conventions	5
2.1	FileName	5
2.2	FunctionName	6
2.3	VariableName	6
2.4	Macros and Constants	7
3	Module Layout	7
3.1	Source Files	7
3.1.1	Template	7
3.1.1.1	Header Comment	7
3.1.1.2	Subroutine Header	8
3.2	Header Files	8
3.3	Template	9
4	Commenting	10
4.1	Block Comments	10
4.2	Single Line Comments	10
4.3	Trailing Comments	11
4.4	Commenting control constructs	11
4.5	Example	11
5	Declarations	12
5.1	Variable Declarations	12
5.1.1	Number Per Line	12
5.1.2	Placement	13
5.2	Function Declarations	14
6	Indentation	14
7	Constants an Macros	15
7.1	Avoid using magic numbers	15
7.2	Use constants and macros instead of hard coded literal values	15
7.3	Only Define constants and macros once	15
7.4	Place parenthesis around each macro parameters	15
8	Statements	15
8.1	return Statements	16
8.2	if, if-else, if else-if else Statements	16
8.3	for Statements	17
8.4	do-while Statements	18
8.5	switch Statements	18
9	Subroutine Guidelines	19
10	Assembly coding for SHARC	21
10.1	Compiler Registers	21
10.2	Call Preserved Registers	21
10.3	Scratch Registers	22
10.4	Stack Registers	23
10.5	Alternate Registers	23
10.6	Alternate Register Usage	24

10.7	Miscellaneous Information About Registers	25
10.8	Transferring Function Arguments and Return Value	25
10.9	Code Release guidelines	26
11	<i>Assembly coding for Blackfin.....</i>	27
11.1	Register Usage Conventions:.....	27
11.1.1	Dedicated Registers	27
11.1.2	Preserved Registers.....	27
11.1.3	Scratch Registers.....	28
11.1.4	Stack registers	28
11.2	Memory Usage by C/C++ Programs	28
11.3	Alignment of assembly, C variables/codes in memory	29
11.4	Calling Assembly Language Subroutines from C Programs and Vice-versa ...	31
11.5	Calling Assembly Language Subroutines from C++ Programs & Vice-versa .	32
11.6	Code release Guidelines.....	33
11.7	General Guidelines.....	33

1 Purpose

The purpose of this document is to provide the guidelines/standard for the Assembly / C / C++ programming.

One of the reasons for developing a coding standard is to make code more readable, which will positively affect the following areas in software development:

- Code generation. A standard reduces the probability of coding error
- Code reviews. A standard increases the efficiency of peer reviews and inspections
- Quality. A standard increases the overall quality of the product
- Maintenance. A standard increases the maintainability of the software product

2 Naming Conventions

➤ Select Clear and Meaningful Names for variables and functions

The most important consideration in naming a variable, constant, or subroutine is that the name fully and accurately describes the entity or action that the structure represents. A clear, complete, and meaningful name makes the code more readable and minimizes the need for comments.

2.1 FileName

The filename should be based on the following conventions

Format:

Modulename_FileName.extension

Description:

- a. Files belonging to the same module should reflect that dependency by a short unique prefix to the file name, followed by an underscore. The module name should be uppercase.
- b. All the words in the file names should start with uppercase; with the extension .h for include files, .c for C source files, .asm for assembly files and .cpp for C++ source files. The file names should reflect the functionality defined/implemented in that file. Files with logical connection (e.g. pairs of header and source files) should reflect that connection in their names wherever possible.

Example:

Transmitter_ISR.asm
Receiver_ISR.asm
Receiver_Autodetect.asm

2.2 FunctionName

The function/subroutine name should be based on the following conventions

Format:

ModuleName_FunctionName ()

Description:

- Functions belonging to the same module should reflect that dependency by a short unique prefix to the file name, followed by an underscore. The module name should be uppercase.
- All the words in the subroutine should start with uppercase

Example:

Receiver_ReadSamplingFrequency (..)

2.3 VariableName

The variable declaration should be done using the variable typedef. The variable name should include type and scope of the variable

Format:

TypeDef VariableTypePrefix_VariableName

The following table explains variable type prefix for variables of different type and scope

Type	Typedef	Variable Type Prefix		Pointer Variable Type Prefix		Array Type Prefix	
		Local Scope	Global Scope	Local Scope	Global Scope	Local Scope	Global Scope
unsigned int	UINT32	u32	gu32	pu32	gpu32	au32	gau32
signed int	SINT32	s32	gs32	ps32	gps32	as32	gas32
unsigned short	UINT16	u16	gu16	pu16	gpu16	au16	gau16
signed short	SINT16	s16	gs16	ps16	gps16	as16	gas16
unsigned char	UINT8	u8	gu8	pu8	gpu8	au8	gau8
signed char	SINT8	s8	gs8	ps8	gps8	as8	gas8
bool	BOOL	b	gb	pb	gpb	ab	gab
float	FLOAT	f	gf	pf	gpf	af	gaf
double	DOUBLE	d	gd	pd	gpd	ad	gad
struct	STRUCT_NAME	st	gst	pst	gpst	ast	gast
enum	ENUM_NAME	e	e	-	-	-	-

Others:

Void Pointer Prefix : pv

Description:
a. Typedef

All the variable declaration should be based on the type-definition mentioned in the above table

b. Variable Type Prefix

All the variables should be prefixed with the variable type followed by the underscore

c. All the words in the Variable name should start with uppercase

Example:

```
UINT32 u32_SamplingFrequency
```

```
UINT32 gu32_SamplingFrequency /*Global Variable*/
```

2.4 Macros and Constants

Fixed identifiers such as Macros and Constants shall contain underscores between words and use all UPPER case letters.

Example:

A constant that defines the maximum frequency of the system could be named “MAX_SYSTEM_FREQUENCY”

3 Module Layout

3.1 Source Files

All the C and C++ source files should have the content in the following order

- 1) Header Comment
- 2) #include statements
- 3) #defines.
- 4) Global Variables Declaration
- 5) Static/File Specific Variables Declaration
- 6) Static/File specific function prototypes.(these functions won't be visible externally) x
- 7) Sub routines
 - o Sub routine header
 - o Implementation

3.1.1 Template

3.1.1.1 Header Comment

Each source file should start with the header comment with the following format

```
/******
```

* Copyright (c) 2006, Jasmin Infotech Private Limited.
 * All rights reserved. Reproduction in whole or part is prohibited without the written permission
 * of the copyright owner.
 *

* Project :
 * Organization :
 * Module :
 * Description :
 *
 *

Change history: CVS will be followed for all the projects. Incorporating the CVS keywords can automate the changes history.

- \$ Revision: \$
- \$ Date: \$
- \$ Author: \$
- \$ Log: \$

*** /

3.1.1.2 Subroutine Header

```
/**
 * Function Name
 *
 * Description
 *
 *Parameters Description
 * Input      : list and meaning of input parameters
 * Output:     : list and meaning of input parameters
 * Modify:     : list and meaning of parameters which is both input and output type
 *
 * @return returns meaning/sample value.
 */
```

3.2 Header Files

All header files should begin with a comment that describes the file and its contents along with the date of creation and a copyright statement.

➤ Ordering
 The header files should have the content in the following ordering

- 1) Header Comment (same as source format)
- 2) #defines.
- 3) Typedefs (structure)
- 4) enums

5) Externally visible functions prototypes.

- To protect against multiple inclusion of headers, the contents of a header file should be protected by a #ifndef ... #endif pair.

```
#ifndef _NAMEOFHEADERFILEINCAPITALS_H_
#define _NAMEOFHEADERFILEINCAPITALS_H_
```

...body of header file...

```
#endif /* _NAMEOFHEADERFILEINCAPITALS_H_ */
```

- As a general rule, header files should not include system header files, but should rather document which system header files they require at the beginning.
- Function prototypes in C header files should be protected against C++ linkage by

```
#ifdef __cplusplus
extern "C"
{
#endif
...prototypes...
#ifdef __cplusplus
}
#endif
```

3.3 Template

```

/*****
* Copyright (c) 2006, Jasmin Infotech Private Limited.
* All rights reserved. Reproduction in whole or part is prohibited without the written permission
* of the copyright owner.
*
* Project      :
* Organisation :
* Module       :
* Description   :
* Change history: CVS will be followed for all the projects. Incorporating the CVS keywords
can automate the changes history.

    • $ Revision: $
    • $ Date: $
    • $ Author: $
    • $ Log: $
*****/

```

```
#ifndef _NAMEOFHEADERFILEINCAPITALS_H_
#define _NAMEOFHEADERFILEINCAPITALS_H_
```

...body of header file...

```
#endif /* _NAMEOFHEADERFILEINCAPITALS_H_ */
```

4 Commenting

- Use `/*...*/`. For commenting the implementation /program
- Comments should contain only information that is relevant to reading and understanding the program. For example, information about how a corresponding package is built or in what directory it resides should not be included as a comment.

Note:

The frequency of comments sometimes reacts poor quality of code. When you feel compelled to add a comment, consider rewriting the code to make it clearer.

4.1 Block Comments

Block comments may be used at the beginning of each file and before each function. They can also be used in other places, such as within functions. Block comments inside a function should be indented to the same level as the code they describe. A blank line to set it apart from the rest of the code should precede a block comment.

```
/*
 * Here is a block comment.
 */
```

4.2 Single Line Comments

Short comments can appear on a single line indented to the level of the code that follows. Normally the double forward slash `//` can be used for a single line comment. If a comment can't be written in a single line, it should follow the block comment.

A blank line should precede a single-line comment.

Here's an example of a single-line comment in C code

```
if (condition)
{
// Handle the condition.
```

```
}
```

4.3 Trailing Comments

Very short comments can appear on the same line as the code they describe, but should be shifted far enough to separate them from the statements. Double forward slash // shall be used for the comment. If more than one short comment appears in a chunk of code, they should all be indented to the same tab setting.(1 tab setting = 4 spaces)

Here's an example of a trailing comment in C code:

```
if (a == 2)
{
a = TRUE;           // special case
}
else
{
a = isPrime(a);     // works only for odd a
}
```

4.4 Commenting control constructs

For constructs (such as “end if”), which close specific, other structures (such as “if Condition”), a comment after the closing construct shall in certain circumstances note the identity of the opening construct. This shall be done if the opening and closing components are textually far apart from one another, or if they are part of a somewhat complex nesting of control structures. This may seem superfluous for short blocks, but is a lifesaver for long and many-nested passages.

```
while (indTotal < 1000)
{

} /* end while indTotal < 1000 */
```

4.5 Example

INCORRECT Method of commenting:

```
/*
   This is the system configuration function
*/
SystemConfig(.....)
{
    ...
};
```

CORRECT Method

```

/*
    This system config function performs the following

    - Configures the PLL.
    - Sets the required interrupt masks. ( SPORT , SPI...)
*/

SystemConfig(.....)
{
    ...
};

```

5 Declarations

5.1 Variable Declarations

5.1.1 Number Per Line

- One declaration per line is recommended since it encourages commenting.

In other words,

```

SINT32 s32_level;           // Indentation levels.
SINT32 s32_attenuate;       // Attenuate value
UINT32 u32_size;           // size of the local buffer.

```

is preferred over

```

SINT32 s32_level, s32_attenuate;

```

- Do not put different types on the same line.

Example:

```

SINT32 s32_level, s32_attenuate [SIZE]; /* WRONG! */

```

Note:

The examples above use one space between the type and the identifier. Another acceptable alternative is to line up a group of declarations (with spaces, not tabs), However line up can be indented to 4 spaces.

e.g.:

```
SINT32    s32_level;           // indentation level
SINT32    s32_attenuate;      // Attenuate value
UINT32    u32_size;           // size of table
```

5.1.2 Placement

- Put declarations only at the beginning of blocks. (A block is any code surrounded by curly braces {,} Don't wait to declare variables until their first use;

```
void MyFunction(...)
{
    SINT32 s32_level;
    s32_level = 0;           // beginning of block

    if (condition)
    {
        SINT32 int2;
        int2 = 0;           // beginning of "if" block
        ...
    }
}
```

- The one exception to the rule is indices of for loops in C++, which can be declared in the for statement:

```
for (UINT32 i = 0; i < MaxLoops; i++)
{
    ...
}
```
- Avoid local declarations that hide declarations at higher levels. For example, do not declare the same variable name in an inner block:

```
UINT32 count;
...
UINT32 MyFunction(...)
{
    if (condition)
    {
        UINT32 count; // AVOID this type of declaration.
        ...
    }
    ...
}
```

5.2 Function Declarations

When coding C functions the following formatting rules should be followed:
All functions should be preceded by a documentation comment describing the function, its arguments and return code(s).

```
/**
 * Function Name
 *
 * Description
 *
 * Parameters Description
 * Input      : list and meaning of input parameters
 * Output:    : list and meaning of input parameters
 * Modify:    : list and meaning of parameters which is both input and output type
 *
 * @return returns meaning/sample value.
 */

SINT32 Sample(SINT32 i, SINT32 j)
{
return i+j;
}
```

6 Indentation

- a) Four spaces should be used as the unit of indentation. Editor will be configured for 1 tab = 4 spaces. Reference to one indentation level means 1 tab (3 space).
- b) Avoid lines longer than 80 characters
- c) When expressions will not fit on a single line, break it according to these general Principles:
 - Break after a comma.
 - Break before an operator.
 - Prefer higher-level breaks to lower-level breaks.
 - Align the new line with the beginning of the expression at the same level on the previous line.

7 Constants and Macros

7.1 Avoid using magic numbers

“Magic numbers” are discouraged. They often make the author’s intent unclear and make global changing of a value undependable.

Wrong:

```
if (fKilnTemperature < 92.46) .....
```

Correct:

```
#define MAX_KILN_TEMP (92.46)

if (fKilnTemperature < MAX_KILN_TEMP) ....
```

7.2 Use constants and macros instead of hard coded literal values

- Literal values shall be avoided in code statements; rather, a symbolic constant for each shall be defined reflecting its intent. 0 should not be assumed to mean “OFF”, a symbolic constant OFF should be defined to be equal to 0.
- Whenever different constants must have fixed relationships and whenever allowed by the language, the fixed relationships shall be forced to hold true. For instance, if ConstantB must be twice the value of ConstantA, define ConstantB as being equal to 2 * ConstantA.

```
#define ConstantB          2*ConstantA
```

7.3 Only Define constants and macros once

Constants and macros shall not be defined in more than one textual location in the program, even if the multiple definitions are exactly the same.

7.4 Place parenthesis around each macro parameters

This will avoid unexpected precedence problems when the macro is expanded into code. C language example: #define PRODUCT (a , b) = ((a) * (b)), not (a * b)

8 Statements

- Braces are used around all statements; even single statements, when they are part of a control structure, such as a if-else or for statement. This makes it easier to add statements without accidentally introducing bugs due to forgetting to add braces.

- The enclosed statements should be indented one more tab than the compound statement. (1 tab = 4 spaces)
- The opening brace should be on a line by itself following the line that begins the compound statement, indented to the same level as that line; the closing brace should begin a line and be indented to the beginning of the compound statement.

IMPORTANT:

All the control statements :- if, while, do-while, for should always use braces {}.

Example:

if statement should always use braces {}. Avoid the following error-prone form:

```
if (condition)                                /*AVOID! THIS OMITTS THE BRACES {}! */
    statement;
```

8.1 return Statements

A return statement with a value should not use parentheses unless they make the return value more obvious in some way.

Example:

```
return;
return MyDisk_size();
return (size ? size : defaultSize);
```

8.2 if, if-else, if else-if else Statements

The if-else class of statements should have the following form:

```
if (condition)
{
    statements; // Indentation level = 1 tab
}
```

```
if (condition)
{
    statements;
}
else
```



```
{  
    statements;  
}
```

```
if (condition)  
{  
    statements;  
}  
else if (condition)  
{  
    statements;  
}  
else  
{  
    statements;  
}
```

8.3 for Statements

A for statement should have the following form:

```
for (initialization; condition; update)  
{  
    statements; // Indentation level = 1 tab  
}
```

An empty for statement (one in which all the work is done in the initialization, condition, and update clauses) should have the following form:

```
for (initialization; condition; update);
```

When using the comma operator in the initialization or update clause of a **for** statement, avoid the complexity of using more than three variables. If needed, use separate statements before the for loop (for the initialization clause) or at the end of the loop (for the update clause).

➤ while Statements

A while statement should have the following form:

```
while (condition)  
{  
    statements; // Indentation level – 1 tab  
}
```

An empty while statement should have the following form:

```
while (condition);
```

8.4 do-while Statements

A do-while statement should have the following form:

```
do
{
    statements; // indentation level = 1 tab
} while (condition);
```

8.5 switch Statements

A switch statement should have the following form:

```
switch (condition)
{
    case ABC:
        statements;
        /* falls through */
    case DEF:
        statements;
        break;
    case XYZ:
        statements;
        break;
    default:
        statements;
        break;
}
```

Every time a case falls through (doesn't include a break statement), add a comment where the break statement would normally be. This is shown in the preceding code example with the `/*falls through */` comment.

Every switch statement should include a default case. The break in the default case is redundant, but it prevents a fall-through error if later the default is changed to a specific case and a new default is introduced.

9 Subroutine Guidelines

➤ Subroutine Scope

Repetitive sections of code should be made subroutines so that parallel maintenance of several copies of the same code can be avoided, and so that maintainers can be sure of the similarity of the passages. Whenever appropriate, sections of code, which are almost the same, except for the identity of some variables, should be made subroutines with parameters to allow for the differing variables.

➤ Subroutine Declaration Guidelines

- Minimize scope by declaring subroutines used only within the module as “static”
- The subroutine’s return type should be declared (do not allow the compiler to select a default value).
- Each parameter type should be declared (do not allow the compiler to select a default value).

➤ Subroutine Layout Guidelines

The specific contents of sections in a subroutine may vary from language to language, but following order should be used as a general guideline for layout of a subroutine:

- Subroutine Header
- Variable declarations
- Comments and Code (intermixed, using “Distributed Code Description”)

➤ Subroutine Size Guideline

Subroutines shall be as long as is necessary to accomplish one independent, cohesive, decoupled function. Size guidelines which limit the number of lines of code, or limit the number of pages of text, are addressing an inconsequential aspect of subroutine design. However, the association between excessive length and the number of independent jobs being performed within a routine is probably a strong one. Review very closely long routines, and judge whether they are performing many tasks that are better parsed out to subordinate subroutines.

➤ Parameter List Guideline

If a subroutine parameter list is longer than one line, lines after the first one will be indented from the left margin so that the second parameter will be listed directly below the first parameter

- The list of the function parameters should have a definite order. The most conventional ordering of parameters is: Input, modified (input and output), and finally output parameters.
- A function that returns information via one or more of its parameters may return only status information in its name
- For complicated function calls (with multiple parameters) listing each parameter passed on a separate line with a short comment describing its function makes the function easier to comprehend.
- For complicated function calls, you can use a prefix for each of the parameters to clearly distinguish input, modify, and output parameters. For example: In_, Mod_, Out_ for input, modify, and output parameters respectively).

10 Assembly coding for SHARC

- The conventions mentioned for C programming should be followed for assembly programming also except C specific guidelines .
- All the assembly programs should be C compliant i.e It should take care of handling processor register carefully.
- “CUSTOMER NAMES should NOT be used in the code for any reason”.
- Code releases from development team to Customer support team should always be NON-TIMERED (ie., default condition should be NON-TIMER). It is the responsibility of the Customer Support team to enable TIMER depending on the customer requirements.

10.1 Compiler Registers

The Compiler cc21k C/C++ run-time environment reserves a set of registers for its own use. The below table lists these registers and the values the C/C++ run-time environment expects to be in them. Do not modify these registers, except as noted in the table.

Register	Value	Modification Rules
m5, m13	0	Do not modify
m6, m14,	1	Do not modify
m7, m15	-1	Do not modify
b6, b7	stack base	Do not modify
l6, l7	stack length	Do not modify
l0, l1, l2, l3, l4, l5, l8, l9, l10, l11, l12, l13, l14, l15	0	Modify for temporary use, restore when done
MMASK (ADSP-2116x/26x/36x processors only)	0xE03003	Do not modify if you are using the interrupt dispatchers supplied with VisualDSP++.

10.2 Call Preserved Registers

The cc21k C/C++ run-time environment specifies a set of registers whose contents must be saved and restored. Your assembly function must save these registers during the function’s prologue and restore the registers as part of the function’s epilogue. These

registers must be saved and restored if they are modified within the assembly function; if a function does not change a particular register, it does not need to save and restore the register.

The table lists these registers.

Call Preserved Registers					
b0	b1	b2	b3	b5	b8
b9	b10	b11	b14	b15	
i0	i1	i2	i3	i5	i8
i9	i10	i11	i14	i15	mode1
mode2	mrB	mrF	m0	m1	m2
m3	m8	m9	m10	m11	r3
r5	r6	r7	r9	r10	r11
r13	r14	r15			

If you use a call preserved I register in an assembler routine called from an assembler routine, you must save and zero (clear) the corresponding L register as part of the function prologue. Then, restore the L register as part of the function epilogue.

Many functions in the C/C++ run-time library expect the processor to be in a specific mode and may not operate correctly if the processor is in a different mode. If you need to change processor modes, save the old values in the mode1 and mode2 registers and restore these registers before calling or returning to calling functions.

The C/C++ run-time environment:

- Uses default bit order for DAG operations (no bit reversal)
- Uses the primary register set (not background set)
- Uses .PRECISION=32 (32-bit floating-point) and .ROUND_NEAREST (round-to-nearest value)
- Disables ALU saturation (mode1 register, ALUSAT bit = 0)
- Uses default FIX instruction rounding to nearest (MODE1 register, TRUNCATE=0)
- Enables circular buffering on ADSP-2116x, ADSP-2126x, ADSP_2136x and ADSP-2137x processors by setting CBUFEN on MODE1

10.3 Scratch Registers

The cc21k C/C++ run-time environment specifies a set of registers whose contents do not need to be saved and restored. Note that the contents of these registers are not preserved across function calls.

The table lists these registers.

Table 1-32. Scratch Registers								
b4	b12	b13	r0	r1	r2	r4	r8	r12
i4	i12	m4	m12	i13	PX	USTAT1	USTAT2	

In addition, for ADSP-2116x processors, the PEy data registers are all scratch registers.

The Table lists these registers.

Table 1-33. Additional ADSP-2116x Scratch Registers								
s0	s1	s2	s3	s4	s5	s6	s7	s8
s9	s10	s11	s12	s13	s14	s15	USTAT3	USTAT4
ASTATy	STKy							

Note: The USTAT registers are now treated as scratch registers

10.4 Stack Registers

The cc21k C/C++ run-time environment reserves a set of registers for controlling the run-time stack. These registers may be modified for stack management, but they must be saved and restored.

The table lists these registers.

Pointer Registers		
Register	Value	Modification Rules
i7	Stack pointer	Modify for stack management, restore when done
i6	Frame pointer	Modify for stack management, restore when done
i12	Return address	Load with function call return address on function exit

10.5 Alternate Registers

The C/C++ run-time environment model does not use any of the alternate registers because these registers are available for use in assembly language only. To use these registers, several aspects of the C/C++ run-time model must be understood.

The C/C++ run-time model uses register I6 as the frame pointer and register I7 as the stack pointer. Setting the DAG register that contains I6 and I7 from a background register to an active register directly affects the stack operation. The C/C++ run-time model does not have an understanding of background registers.

If the background I6 and I7 registers are active and an interrupt occurs, the C/C++ run-time model still uses I6 and I7 to update the stack. This results in faulty stack handling.

Note: The background register set containing DAG registers I6 and I7 should only be used in assembly routines if interrupts are not enabled.

The super fast interrupt dispatcher uses context switching rather than saving registers on the run-time stack. To ensure no register conflicts, do not use the super fast interrupt dispatcher or disable interrupts when using secondary registers in an assembly routine.

10.6 Alternate Register Usage

SHARC processors provide a set of alternate (or background) registers for both the computation units and the DAGs. VDK does not save and restore these alternate registers during context switches but instead employs them to accelerate entry to and exit from the scheduler interrupt by providing it with a dedicated C run-time environment.

This means that the alternate registers listed in below Table contain fixed values and must not be changed by user code.

SHARC DSP Alternate Registers	
Register	Value
M5', M13'	0
M6', M14'	1
M7', M15'	-1
B6', B7'	System Stack base
L6', L7'	System Stack length
L2'—L5', L10'—L15'	0

Since these registers are used at Interrupt Level (which may be entered at any time), it is not sufficient to save these registers and restore them after use, except within a critical region.

The following alternate registers are not used by VDK and are available for use by user code:

I0', M0', B0', L0',
 I1', M1', B1', L1',
 I8', M8', B8', L8',
 I9', M9', B9', L9',

Note that these registers do not form part of the thread context and, hence, are not context-switched between threads. Therefore, their most appropriate use is in implementing fast I/O interrupt handlers for devices where DMA is not available.

The remaining alternate registers:

R0'—R15'
 I2'—I7', I10'—I15',
 M2'—M4', M10'—B12',
 B2'—B5', B10'—B15'

are used by the scheduler ISR and, hence, may change at any time (unless inside a critical region). If these registers are to be used in a user-defined ISR, then their contents must be saved and restored before returning from the interrupt, as for the primary registers. Note that the background multiplier result register, MRB, is not considered to be an alternate register in the above discussion, and that both MRB and MRF are context-switched between threads.

10.7 Miscellaneous Information About Registers

The following is some miscellaneous information that you might find helpful in understanding register functionality:

- All of the L registers, except L6 and L7, are required to be zero at any call/return point.
- When you either make a function call or return to your caller and have modified any of the L registers, you must reset them to zero.
- Interrupt routines must save and set to zero the L register before using its corresponding I register for any post-modify instruction.
- The MMASK register ensures that MODE1 is set to the correct value before the interrupt dispatcher code is executed. It ensures that the following bits are cleared: BR0, BR8, IRPTEN, ALUSAT, PEYEN, BDCST1, BDCST9.

10.8 Transferring Function Arguments and Return Value

The C/C++ run-time environment uses a set of registers and the run-time stack to transfer function parameters to assembly routines. Your assembly language functions must follow these conventions when they call or when they are called by C or C++ functions.

Because it is most efficient to use registers for passing parameters, the run-time environment attempts to pass the first three parameters in a function call using registers; it then passes any remaining parameters on the run-time stack.

The convention is to pass the function's first parameter in r4, the second parameter in r8, and the third parameter in r12. The following exceptions apply to this convention:

- If any parameter is larger than a single 32-bit word, then that parameter and all subsequent parameters are passed on the stack.
- If the function is declared to take a variable number of arguments (has ... in its prototype), then the last named parameter and any subsequent parameters are passed on the stack.

The table lists the rules that cc21k uses for passing parameters in registers to functions and the rules that your assembly code must use for returns.

Parameter and Return Value Transfer Registers	
Register	Parameter Type Passed Or Returned
r4	Pass first 32-bit data type parameter
r8	Pass second 32-bit data type parameter
r12	Pass third 32-bit data type parameter
stack	Pass fourth and remaining parameters;
r0	Return int, long, char, float, short, pointer, and one-word structure parameters
r0, r1	Return long double and two-word structure parameters. Place MSW in r0 and LSW in r1
r1	Return the address of results that are longer than two words; r1 contains the first location in the block of memory containing the results

10.9 Code Release guidelines

- CUSTOMER NAMES should NOT be used in the code for any reason.
- Code releases from development team to Customer support team should always be NON-TIMERED (i.e., default condition should be NON-TIMER). It is the responsibility of the Customer Support team to enable TIMER depending on the customer requirements.

11 Assembly coding for Blackfin

- The conventions mentioned for C programming should be followed for assembly programming also, except C specific guidelines.
- All the assembly programs should be C compliant i.e. it should take care of handling processor register carefully.

11.1 Register Usage Conventions:

Based on the usage of registers by C or C++ compiler the Blackfin DSP registers are separated in to following categories

- Dedicated Registers
- Preserved Registers
- Scratch Registers
- Stack Registers

11.1.1 Dedicated Registers

The Blackfin C environment specifies a set of registers whose contents should not be changed except in specific defined circumstances. If these registers are changed then their values must be saved and restored. Table 2-1 lists the various categories of Blackfin DSP registers.

Dedicated registers	Preserved Registers	Scratch registers	Stack registers
SP,FP L0,L1,L2,L3	P3,P4,P5 R4,R5,R6,R7	P0,P1,P2 R0,R1,R2,R3 LC0,LC1 CC A0,A1 I0,I1,I2,I3 M0,M1,M2,M3 B0,B1,B2,B3	SP,FP

11.1.2 Preserved Registers

The Blackfin C Compiler specifies a set of registers whose contents must be saved and restored. The assembly function must save these registers during the function's prologue and restore the registers as part of the function's epilogue. These registers must be saved and restored if they are modified within the assembly function; if a function does not change a particular register, it does not need to save and restore the register.

11.1.3 Scratch Registers

The Blackfin C/C++ Compiler specifies a set of registers whose contents do not need to be saved and restored in the assembly function. Note that the contents of these registers are not preserved across function calls.

11.1.4 Stack registers

The Blackfin C/C++ run-time environment reserves a set of registers for controlling the run-time stack.

Note:

1.The C run-time model defines some registers as scratch registers and others as preserved or dedicated registers as explained above. The scratch registers can be used within the assembly language program without worrying about their previous contents

2.The preserved registers can also be used in the assembly routines. But the contents of these registers must be saved in entry and restored at the exit of assembly routine.

3.Do not use the dedicated registers for other than their intended purpose.

11.2 Memory Usage by C/C++ Programs

The compiler puts all the codes and datas in to the following memory section by default.

- The code memory section, “[program](#)”, is where the compiler puts all the program instructions that it generates when it compiles the program.
- The data memory section “[data1](#)”, is where the compiler puts global and static data in memory. The data memory section, [constdata](#), is where the compiler puts data that has been declared as “const”.
- The above memory sections should be defined in the Linker Description File (LDF). If the user wants to place the C/C++ codes and datas in to some other memory section then that memory section name should be mentioned on the top of each and every subroutine, variables.

Example1 :

```
Segment ("Program_Application")
Void TestFunc( )
{
}
```

The compiler places all the codes of TestFunc subroutine in to “[Program_Application](#)” memory section.

Example2:

```
Segment ("Data_Application")
Unsigned char CodecSampFreq;
```

The compiler places the variable CodecSampFreq in the “[Data_Application](#)” memory section

- The Blackfin C/C++ run-time environment uses the stack memory for storage of
 - Automatic variables/local variables of C programs
 - Return addresses.
 - Storing and Restoring of Preserved Registers.

So the sufficient Stack Memory Space should be allocated based on the number of Automatic variables/local variables used by the C Programs

11.3 Alignment of assembly, C variables/codes in memory

The BlackFin DSP processor supports 32bit, 16bit, and 8bit accessing of memory. So we should place the 32 bit, 16bit, 8 bit variables at the correct boundary location in memory. So to avoid the **data miss aligned access**, the variables should be placed at the memory in following manner.

- The 32-bit variable should be placed in the 4byte-boundary memory location. i.e the variable should be placed at the memory location with the address value that can be evenly divided by 4
- The 16 bit variable should be placed in the 2byte-boundary memory location. i.e the variable should be placed at the memory location with the address value that can be evenly divided by 2.
- The 8 bit variable should be placed in the 1byte-boundary memory location. i.e the variable should be placed at the memory location with the address value that can be evenly divided by 1. So we can place the 8 bit variable in any memory location.
- The codes can be placed in 4 byte boundary memory location.
- The aligning of memory variables/codes can be done using **“.ALIGN”** assembler directive.
- The **“. ALIGN”** directive forces the address alignment of an instruction or data item within the. SECTION it is used. The **“. ALIGN”** directive uses the following syntax.

.ALIGN expression ;

Where: expression evaluates to an integer. The expression specifies the byte alignment requirement; its value must be a power of 2. When aligning a data item or instruction, the assembler adjusts the address of the current location counter to the next address so it can be evenly divided by the value of expression , or aligned. The expression set to 0 or 1 signifies no address alignment requirement.

In the absence of the .ALIGN directive, the default address alignment is set to 1.

Example1 :

```
.ALIGN 4;
/*declaration of 32 bit variable*/
.var variable1;
```

Aligns the first data on a 32 bit boundary and advances other data items

```
.var variable2;
.var variable3;
```

```
.ALIGN 2;
/*declaration of 16 bit variable*/
.byte2 variable1;
.byte2 variable2;
.byte2 variable3;
```

Aligns the first data on a 16 bit boundary and advances other data items

No alignment required here

```
/*declaration of 8 bit variable*/
.byte variable1;
.byte variable2;
.byte variable3;
```

For proper alignment of variables, group the related variables together i.e., declare the entire 32 bit variables first, followed by 16 bit and finally 8 bit variables.

Example 2:

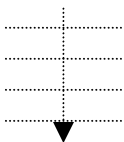
The same procedure can be followed at declaration of C variables also. The assembly directive `.ALIGN4` can be placed in the C routines by using `asm()` assembly construct.

```
asm ( ".ALIGN 4; " );
int variable1;
```

Example 3:

The Instructions/codes can be placed in 4 byte boundary memory location in the following manner

```
AsmFunc:
.global _AsmFunc;
.ALIGN 4;
[ --SP ] = RETS;
[ --SP ] = (R7:4, P5:3);
[ --SP ] = L0;
[ --SP ] = L1;
[ --SP ] = L2;
[ --SP ] = L3;
[ --SP ] = FP;
```



11.4 Calling Assembly Language Subroutines from C Programs and Vice-versa

The following procedure should be done in the assembly routine for calling it from C programs

- Append a leading underscore (_) at the subroutine label and declare it as global.
- Store the contents of dedicated, preserved registers at the entry of subroutine. The contents of these registers can be pushed to the stack
- The parameters always in the R0,R1,R2, Stack .So read necessary input parameters
- Store the return value/result in to the R0-R1 pair based on their sizes
- Restore the contents of dedicated, preserved registers if it is saved at the Subroutine Entry

In the C routine, we should declare this assembly subroutine with appropriate arguments.

Example:

An assembly function called “_AsmFunc” has two parameters. These parameters are 32 bit wide and contains unsigned magnitude value. This function returns 32 bit result and it is also a unsigned number. The following figure explains the calling of this asm function from C.

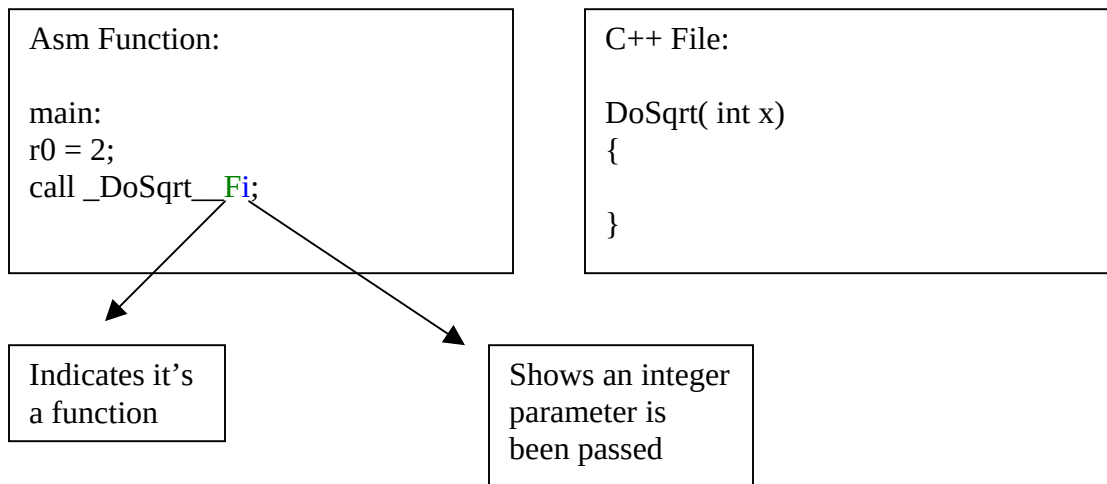
C Function	Assembly Subroutine
<pre> /*declare asm function prototype */ extern unsigned int AsmFunc(unsigned int , (unsigned int); /* C Function that calls asm Function */ void Cfunction() { int Parameter1; int Parameter2; int ReturnValue; ----- ----- /* calling of assembly function */ ReturnValue = AsmFunc(Parameter1,Parameter2); ----- ----- </pre>	<pre> /*declare asm function*/ _<u>AsmFunc</u>: .global AsmFunc; /*Push Dedicated ,Preserved registers modified by this routine in to the stack*/ [--SP] = (R7:4,P5:3); //preserved register /* Read the Input parameter from R0,R1 ,R2 register. i.e, the arguments are available in the R0,R1 register when it is called from C Function*/ /* Do processing */ /* store the result/return value to the R0 register. The return value should be in R0 register while passing return value to C function */ /*Restore dedicated register/preserved register content*/ (R7:4,P5:3) = [SP++]; //preserved registers RTS; </pre>

The same procedure to be followed while calling C function from assembly also.

11.5 Calling Assembly Language Subroutines from C++ Programs & Vice-versa

- Calling assembly functions from C++ follows the same rule as C.
- Care has to be taken while calling C++ functions from C or Assembly.
- The compiler demangles the function name and adds information related to arguments passed and return parameters.

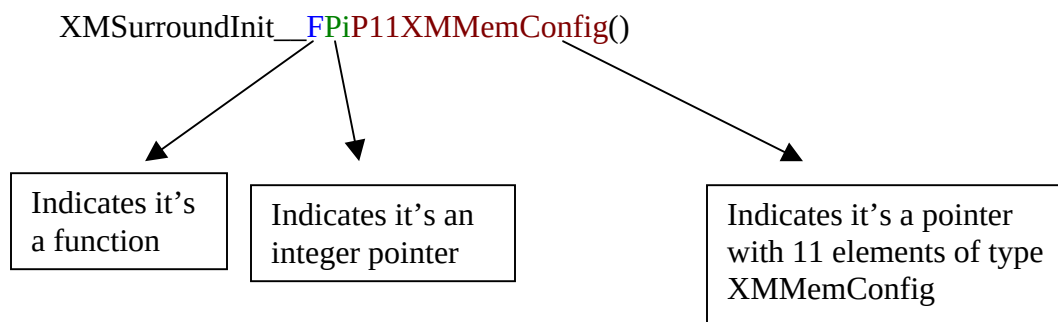
Example 1:



Following function is been defined in C++

XMSurroundInit(int *Input,XMMemConfig *memptr)

And its demangled name will be as follows



11.6 Code release Guidelines

- CUSTOMER NAMES should NOT be used in the code for any reason.
- Code releases from development team to other support teams should always be NON-TIMERED (i.e., default condition should be NON-TIMER). If required, timer will be added to the codes.
- The code should always be encrypted before release.

11.7 General Guidelines

- Need to add .end to all the assembly modules.
- Need to use a dedicated section for code and data For example, like "DDPlusCode", "DDPlusDataExternal" etc instead of default sections - data1 and program.
- Need to define proto types for assembly functions called in C also.
- There should be no dynamic allocation of memory inside the library.
- All other memory should be declared in the kernel and the memory pointers should be passed to the library. Library should allocate the all the buffers and variables in this memory.
- Can define a general structure from the kernel to pass to the library like,

For example,

Typedef struct

```
{
int *InputBufRdPtr;
int *InputBufferBase;
int InputBufferLength;
int *OutputBufWrPtr;
int *OutputBufferBase;
int OutputBufferLength;
struct *InputParam;
struct *OutputParam;
int *MemoryPtr1;
int * MemoryPtr2;
int * MemoryPtr3;
}handle;
```

struct inputparam

```
{
can include the controls which the user like to vary
}
```

struct outputparam

```
{
can include the variables which the user like to know like
```

```
int Sampling frequency
```

```
int Bit rate
int Number of channels
int error status
.....

}
```